Final Project - Intro to Data Science

2026-09-29

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(dplyr)
```

1. **Downloaded data**

```
# Explanation: I want to load the count data. I also want to generate some
# initial variables that I expect to use across code blocks.

tcga_data <- read.csv(file = "counts.csv", row.names = 1) # Read data from
#"counts" CSV


esr1_gene <- tcga_data["ENSG00000091831",] # Instantiate expression values
# for my gene of choice

esr1_numeric <- as.numeric(esr1_gene) # Instantiate numeric vector of
# expression values for use throughout interpretations, as needed

sum(is.na(esr1_numeric)) # Check if there are any NAs that we'd want to manage
```

```
## [1] 0
```

```
her2_gene <- tcga_data["ENSG00000141736",] # Instantiate expression
#values for my other gene of choice

her2_numeric <- as.numeric(her2_gene) # " " re: expression values

sum(is.na(her2_gene)) # " " re: NAs
```

```
## [1] 0
```

2) **Summary Statistics**

```
# Explanation: I want generate summary statistics in a tidy way

summary_stats <- tibble(expression = esr1_numeric) %>% # Create data frame
```

```
  # of numeric vector
  summarize( # Generate summary statistics with summarize() function
    mean = mean(expression, na.rm = TRUE),
    median = median(expression, na.rm = TRUE),
    sd = sd(expression, na.rm = TRUE),
    min = min(expression, na.rm = TRUE),
    max = max(expression, na.rm = TRUE)
  )

summary_stats # Return the statistical data
```

```
## # A tibble: 1 x 5
##     mean median     sd   min    max
##    <dbl>  <dbl>  <dbl> <dbl>  <dbl>
## 1 31442.  17561 38261.    14 270977
```

3) **Visualization**

```
# Explanation: I want a function to create the histogram, scatter, and violin plots.
# Since each plot function has a lot of similar data, and we're only working in
# this file, we can create a function that generates all 3 plots.

library(ggpubr) # Since not part of the tidyverse, and we need ggarrange() below

create_plots <- function(first_gene_expression,
                         first_gene_name,
                         second_gene_name,
                         second_gene_numeric,
                         metadata,
                         fill_or_color = "#1B9E77") {

  # Because we want to make a new vector from the argument passed
  first_gene_numeric <- as.numeric(first_gene_expression)

  # Histogram
  hist_plot <- ggplot(
    data.frame(expression = first_gene_numeric),
    aes (x = expression)
  ) +

    geom_histogram(bins = 30,
                   alpha = 0.8,
                   fill = fill_or_color,
                   color = "black") +
    labs(
      title = paste("Distribution of", first_gene_name, "Expression"),
      x = "Gene explression value",
      y = "Frequency"
      ) +
    theme_minimal() +
    theme(plot.margin = margin(15, 15, 15, 15), # I want the plot to have
          # padding
          plot.title = element_text(face = "bold")) + # Embolden plot title
    scale_x_continuous(breaks = seq(0, 300000, by = 50000))
```

```r
# Scatter plot
scatter_df <- data.frame(
  Gene_1 = first_gene_numeric,
  Gene_2 = second_gene_numeric
)

scatter_plot <- ggplot(
  scatter_df,
  aes(x = Gene_1, y = Gene_2)
) +
  geom_point( # Apply scatter geometry
  color = fill_or_color,
  alpha = 0.6
) +

# Since the expression ranges are so large, I want to log them in order
# to visualize/interpret the data more efficiently
scale_x_log10(
  breaks = c(10, 100, 1000, 10000, 100000),
  labels = c("10", "100", "1,000", "10,000", "100,000")
) +
scale_y_log10(
  breaks = c(1000, 10000, 100000, 1000000),
  labels = c("1,000", "10,000", "100,000", "1,000,000")
) +
labs(
  title = paste(first_gene_name, "vs", second_gene_name, "Expression Across Patient Samples"),
  x = paste(first_gene_name, "Expression (log10 scale)"),
  y = paste(second_gene_name, "Expression (log10 scale)")
) +
theme_minimal() +
theme(plot.margin = margin(15, 15, 15, 15),
      plot.title = element_text(face = "bold"))


# Violin Plot
violin_data_frame <- tibble(

  # head(names(esr1_gene)) returns samples with a period separator. In
  # order to match the metadata barcodes, we have to turn each period
  # into a hyphen
  barcode = gsub("\\.", "-", names(first_gene_expression)),
  expression = first_gene_numeric
  ) %>%
  inner_join(
    metadata %>%
    select(barcode, ajcc_pathologic_stage),
    by = "barcode"
    ) %>%

  # The data have some NAs and X's in the ajcc_pathologic_stage column,
  # which we want to filter out
  filter(!is.na(ajcc_pathologic_stage),
```

```
          ajcc_pathologic_stage != "Stage X")

  violin_plot <- ggplot(violin_data_frame,
                        aes(x = ajcc_pathologic_stage,
                            y = expression)) +
    geom_violin(fill = fill_or_color,
                alpha = 0.7) +
    scale_y_log10(
      breaks = c(10, 100, 1000, 10000, 100000),
      labels = c("10", "100", "1,000", "10,000", "100,000")
      ) +
    labs(title = paste(first_gene_name, "Expression by Pathologic Stage"),
         x = "Pathologic Stage",
         y = paste(first_gene_name, "Expression (log10 scale)")) +
    theme_minimal() +
    theme(
      axis.text.x = element_text(
        angle = 45,
        hjust = 1,
        size = 8),
      plot.margin = margin(15, 15, 15, 15),
      plot.title = element_text(face = "bold"))

  # Arrange plots
  all_plots <- ggarrange(hist_plot, scatter_plot, violin_plot,
                         ncol = 2,
                         nrow = 2,
                         labels = c("3A", "3B", "3C"))

  return(all_plots)
}
```

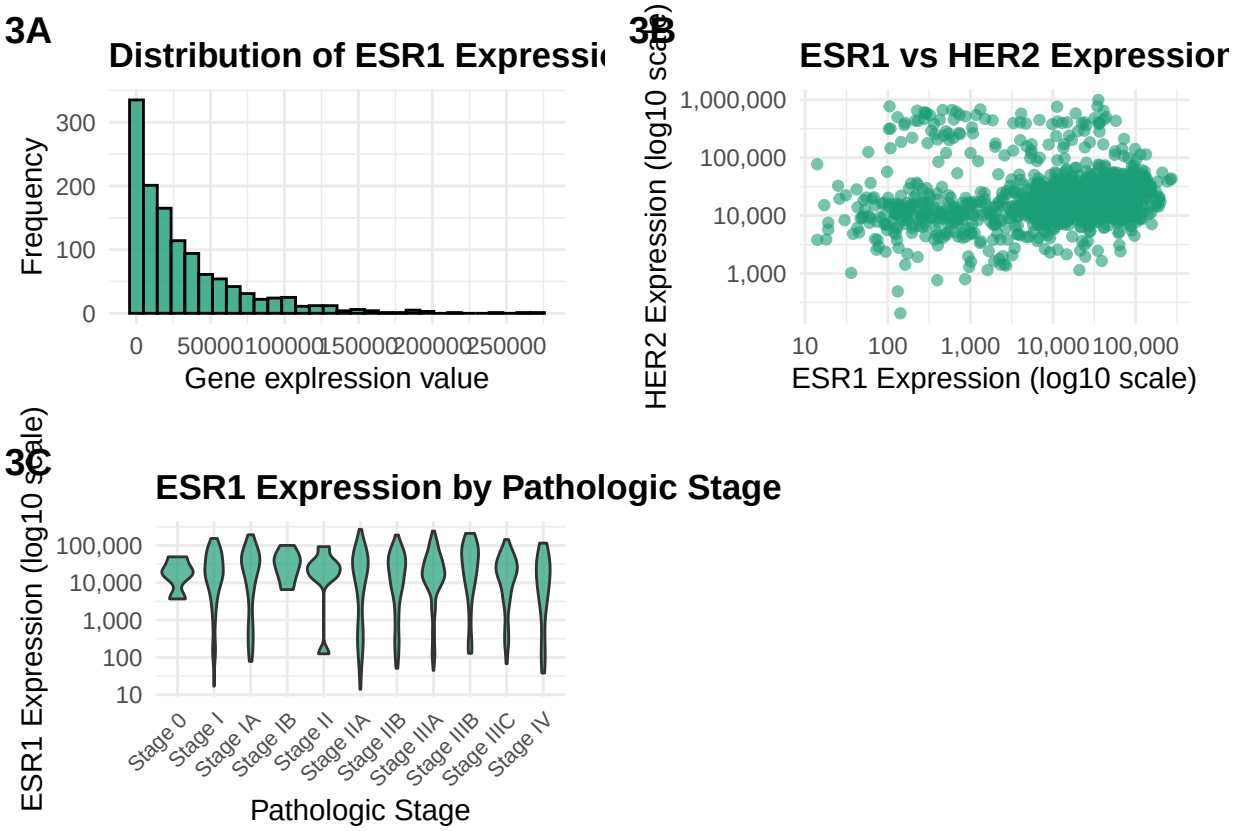
#'s 3A. through 3C.

```
# Explanation: Now, I need to call the function with the data for each plot to
# generate the final plots. Thereafter, I save the plots to a PNG.

# I need to fetch the metadata to generate the violin plot
tcga_meta_data <- read.csv(file = "meta_data.csv")

final_plots <- create_plots(
  first_gene_expression = esr1_gene,
  first_gene_name = "ESR1",
  second_gene_name = "HER2",
  second_gene_numeric = her2_numeric,
  metadata = tcga_meta_data
)

final_plots
```




```
ggsave(
  filename = "SalaamRasheeda-Final Project Check-In Plots.png",
  plot = final_plots,
  width = 12,
  height = 8,
  dpi = 300
)
```

3D. Heatmap Analysis

```
# Explanation: We have over 1,000 samples, so the heatmap will look busy. I've
# done my best to allow for some readbility by structuring the covariate colors
# to separate into broader pathologic stages.

# load heatmap libraries
library(ComplexHeatmap)
```

```
## Loading required package: grid
```

```
## ========================================
## ComplexHeatmap version 2.28.0
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##     genomic data. Bioinformatics 2016.
```

```
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## ========================================
```

```
library(circlize)
```

```
## ========================================
## circlize version 0.4.18
## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize_book/book/
##
## If you use it in published research, please cite:
## Gu, Z. circlize implements and enhances circular visualization
##   in R. Bioinformatics 2014.
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(circlize))
## ========================================
```

```
# Select 10 consecutive genes, starting from ENSG00000091831-- my chosen
# ESR1 gene
which(rownames(tcga_data) == "ENSG00000091831.24")
```

```
## [1] 1944
```

```
ten_genes <- rownames(tcga_data)[1944:1953]

# Filter covariate data to exclude NAs and X's
filtered_tcga_meta_data <- tcga_meta_data %>%
  filter(
    !is.na(ajcc_pathologic_stage),
    ajcc_pathologic_stage != "Stage X"
  )

# In counts data, replace period delimiter with hyphens
count_barcodes <- gsub("\\.", "-", colnames(tcga_data))

# Grab matching rows from filtered data to create subset below
samples_to_keep <- filtered_tcga_meta_data$barcode

# Subset these 10 genes into a data frame
tcga_data_subset <- tcga_data[
  ten_genes,
  count_barcodes %in% samples_to_keep
  ]

# Get log of expression values for better readability for Heatmap() below
log_counts <- log10(tcga_data_subset + 1)

# Create heatmap annotations
```

```r
ha_column <- HeatmapAnnotation(
  Pathologic_Stage = filtered_tcga_meta_data$ajcc_pathologic_stage,
  col = list(
    # Create a color palette that helps to visualize the broader stages,
    # since we have so many samples
    Pathologic_Stage = c(
      "Stage 0" = "#000000",

      "Stage I" = "#66C2A5",
      "Stage IA" = "#41AE76",
      "Stage IB" = "#238B45",

      "Stage II" = "#8DA0CB",
      "Stage IIA" = "#4F81BD",
      "Stage IIB" = "#225EA8",

      "Stage IIIA"= "#FC8D62",
      "Stage IIIB"= "#F16913",
      "Stage IIIC"= "#D94801",

      "Stage IV" = "#E7298A"
      )
    )
  )

# Save heatmap to PNG with adequate dimensions
png(
"SalaamRasheeda-Final Project Check-in Heatmap.png",
width = 16000,
height = 8000,
res = 1200
)

# Generate heatmap
Heatmap(
  as.matrix(log_counts), # Because Heatmap() expects data in a matrix
  name = "Counts",
  top_annotation = ha_column,
  show_row_names = TRUE,
  show_column_names = FALSE, # We have 1,000+ samples, so showing the
  # names would be very messy
  cluster_columns = TRUE,
  cluster_rows = TRUE,
  column_title = "Samples",
  row_title = "Genes",
  heatmap_legend_param = list(title = "Count")
)

dev.off()
```

```
## pdf
##   2
```